

Assignment: Project Management REST API using Node.js, Express.js and MongoDB

1. Problem Statement

You are required to develop a **Project Management Backend Application** using **Node.js, Express.js, MongoDB, and Mongoose**.

The application should manage three main entities/Database Models:

1. **Users**
2. **Projects**
3. **Tasks**

The Project must be developed using **MVC Architecture**, with proper separation of responsibilities between routes, controllers, services/business logic, models, and other required layers.

The application should provide **separate CRUD operations for each entity/Database Model** and should correctly handle the relationships between Users, Projects, and Tasks.

2. Data Models

The application must contain the following three MongoDB collections/models.

2.1) User

Sample fields/hints:

name

userId

email

role

experience

2.2) Project

Sample fields/hints:

projectName

projectId

managerId

ownerId

createdBy

updatedBy

2.3) Task

Sample fields/hints:

taskName

taskId

projectId

userId

Note: The above fields are provided only as a reference/hint. You can use or change by your own.

3) Relationships

3.1) Project → User

A project contains references to users through:

managerId

ownerId

createdBy

updatedBy

All these IDs should refer to existing records in the **Users** collection.

3.2) Task → Project

A task belongs to a project through:

projectId

The provided projectId must refer to an existing record in the **Projects** collection.

3.3) Task → User

A task is assigned to a user through:

userId

The provided userId must refer to an existing record in the **Users** collection.

4. CRUD Operations

You must implement **separate CRUD operations for each model**.

User CRUD

Implement APIs for:

Create User

Get All Users

Get User by ID

Update User

Delete User

Project CRUD

Implement APIs for:

Create Project

Get All Projects

Get Project by ID

Update Project

Delete Project

When creating or updating a Project, the request payload will contain User IDs for fields such as:

managerId

ownerId

createdBy

updatedBy

Before creating or updating the Project, the application must verify that the referenced User records actually exist in the **Users collection**.

For example:

Create Project Request



Receive managerId



Check managerId in Users collection



Does User exist?



YES



Continue



NO



Reject request

The same validation must be performed for every User reference being supplied.

Task CRUD

Implement APIs for:

Create Task

Get All Tasks

Get Task by ID

Update Task

Delete Task

When creating or updating a Task, the request payload will contain:

projectId

userId

Before creating or updating the Task:

projectId

Verify that the referenced Project exists in the **Projects collection**.

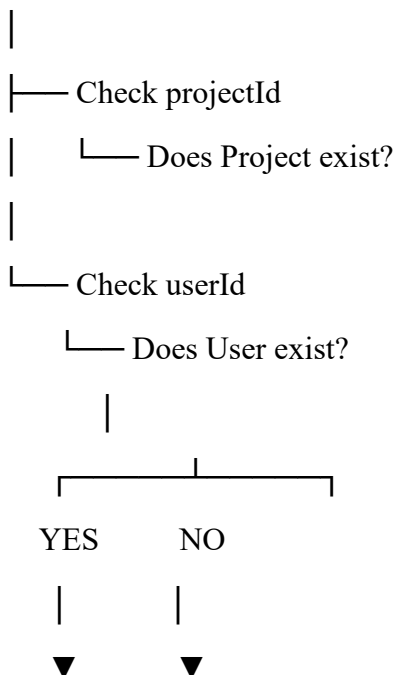
userId

Verify that the referenced User exists in the **Users collection**.

The Task should only be created or updated if **both referenced records exist**.

Example:

Create Task



5. Architecture Requirement

The application must **not** be implemented as a single large Express file.

Use:

MVC Architecture

Routes



Controllers



Services



Models



MongoDB

The folder structure is your responsibility, but the code should have a clear separation of concerns.

For example:

src/

|

|— models/

|— routes/

|— controllers/

|— services/

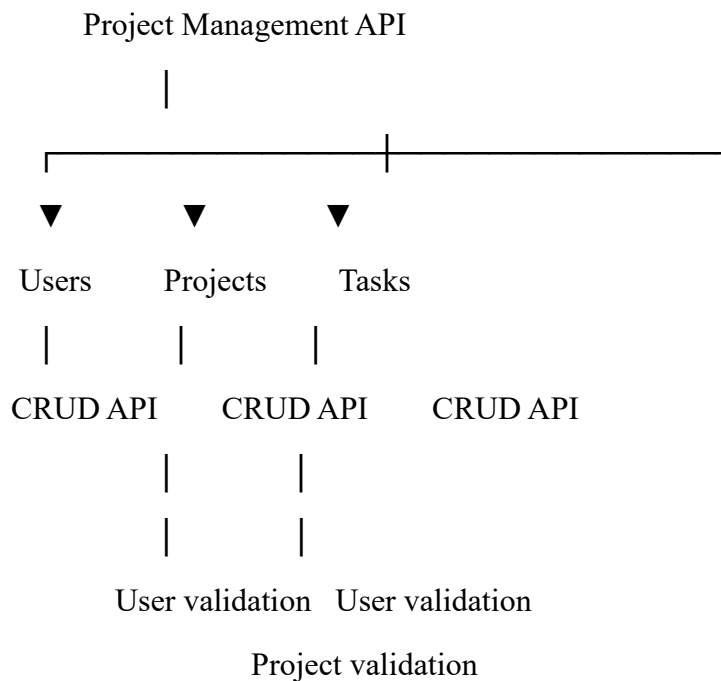
|— repositories/

|— middlewares/

|— validators/
|— config/
|— app.js
|— server.js

12. Expected Outcome

By completing this assignment, the application should provide:



The main objective is not only to make the CRUD APIs work, but to demonstrate that you understand:

- Express routing
- REST API design
- Mongoose models
- MongoDB relationships/references
- CRUD operations
- Request validation
- Referential validation

- MVC/layered architecture
- Separation of concerns
- Proper error handling
- Environment configuration
- Production-oriented project organization